

The background image depicts a dark, atmospheric scene with several tall, jagged, stone-like pillars. Each pillar is illuminated with a series of horizontal blue light bands. A central, taller pillar is topped with a bright, glowing blue ring of light, from which several thin, curved blue lines of light extend to the other pillars. The ground is dark and reflective, mirroring the blue light. The sky is dark with faint stars and a hazy, blue-tinged atmosphere.

SISTEMAS DISTRIBUIDOS

CLASE 2 — MAPREDUCE

Hoja de ruta

1. ¿De dónde viene MapReduce? Cinco eras de sistemas distribuidos
2. ¿Qué problema resuelve?
3. ¿Cómo se pasa del modelo a un clúster real?
4. ¿Qué hacen por dentro un mapper y un reducer?
5. ¿Quién reparte el trabajo y por dónde viajan los datos?
6. ¿Qué pasa cuando algo falla?

CINCO ERAS DE SISTEMAS DISTRIBUIDOS

Era 1: los fundamentos teóricos (70s–80s)

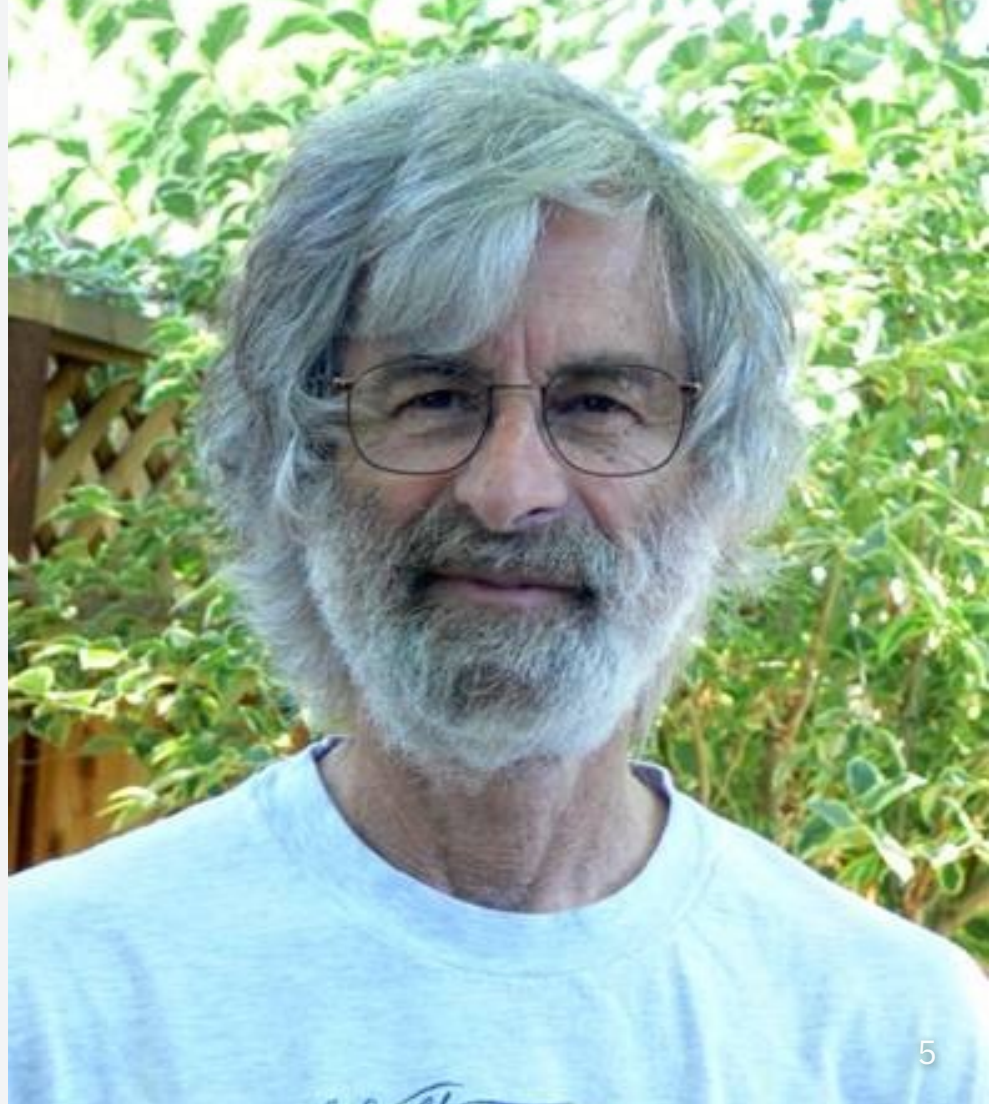
- Fundamentos matemáticos de los sistemas distribuidos.
- **¿Cómo se coordinan procesos que no comparten memoria ni reloj?**

Ninguno de los dos recursos está compartido en un sistema distribuido. El resto de la materia son técnicas que mitigan esa ausencia.

Leslie Lamport

- Escribió **LaTeX**, sobre el TeX de Knuth: la herramienta con la que se escriben los papers
- Y **TLA+**, un lenguaje para especificar sistemas y verificar que son correctos

De ahí surge su primera observación: la imposibilidad de sincronizar relojes también aparece en los sistemas distribuidos.



Relojes lógicos y relojes vectoriales

- El nombre engaña: **no** resuelven el problema de tener un reloj global
- Sirven para afirmar con certeza que un evento ocurrió **después** que otro
- Es un orden **relativo**, no absoluto

Lo que se resuelve con eso tiene nombre: la **causalidad**.

La máquina de estados replicada

- Tolerancia a fallas por **réplicas**: en vez de una base de datos, dos iguales
- Una base de datos es una máquina de estados: dado un evento, se sabe a qué estado va
- Mismo estado inicial + mismas operaciones **en el mismo orden** $\Rightarrow$ mismo estado final

La cuenta bancaria: partiendo del saldo de hace un mes y aplicando todas las transacciones en orden, se llega al saldo de hoy. Funciona porque cada operación es **determinista**.

El problema se corrió al orden

- Aparentemente resolvimos la replicación; en realidad **trasladamos el problema**
- Ahora la pregunta es cómo llega el mismo orden a las dos máquinas
- Una única máquina que defina el orden funciona, pero si esa máquina falla, falla todo
- Hace falta que las máquinas **se pongan de acuerdo** en cuál es la próxima operación

Eso es el problema de **consenso** → **Paxos** (fines de los 80) → **Raft** (2014)

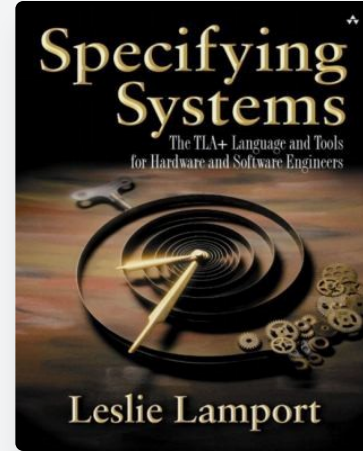
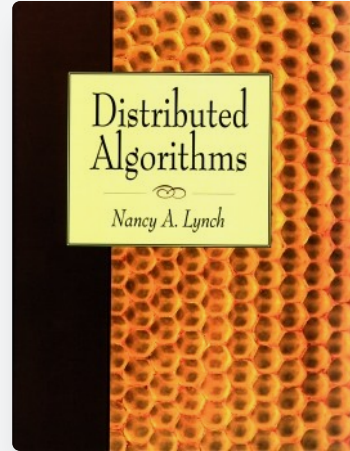
Nancy Lynch

- **FLP** (1985): Fischer, **Lynch**, Paterson
- Con una sola máquina que pueda caerse, **el consenso no tiene solución determinista**
- Vale en un sistema **asincrónico**, y está **demostrado**: no es un problema abierto

Paxos y Raft resignan **terminar siempre** a favor de la corrección.



Referencias bibliográficas



Nancy A. Lynch, *Distributed Algorithms* (Morgan Kaufmann, 1996) • Leslie Lamport, *Specifying Systems* (Addison-Wesley, 2002)

Ninguno de los dos es bibliografía obligatoria de la materia. El de Lamport, además, **se baja gratis de su sitio.**

Era 2: la transparencia (80s–90s)

- La estrategia, en una línea: **hacer un sistema distribuido es ocultar la red**

Ejemplos

- **NFS** — funcionaba bien cuando funcionaba bien
- **RPC** — el que mejor sobrevivió: se quedó con la comodidad y abandonó el engaño
- **SO distribuidos** — Amoeba (Tanenbaum) y Plan 9 from Bell Labs

Lo que quedó de Plan 9

- **UTF-8** — Thompson y Pike lo diseñaron en 1992 mientras convertían Plan 9 a Unicode
- **9P** — su protocolo de archivos, implementado en Linux y en WSL
- `rfork` — la base de los threads livianos; en Linux reapareció como `clone`
- **Go** — dos de sus tres diseñadores, Thompson y Pike, venían de ahí

Waldo et al., 1994

A Note on Distributed Computing

- No hay que tratar de esconder la red
- Cuatro propiedades que no se pueden ocultar: **latencia, acceso a memoria, concurrencia y fallas parciales**

Era 3: el middleware (90s)

- La época de la **computación corporativa**
- El problema **no** era el volumen de datos: *big data* todavía no se usaba como término
- El problema era la **comunicación** entre sistemas

La decisión conceptual de la era, que no se deshizo nunca más: se descarta ocultar completamente la red y se le da una **capa propia**.

Middleware

- **Comunicación** — el problema original
- **Discovery** — con quién hay que conectarse y cómo se llama
- **Transmisión de datos**
- **Manejo de errores** — donde la comunicación pasa por una red, es una responsabilidad enorme

Ejemplos de Middlewares: CORBA, RMI y REST

- **CORBA** — *Common Object Request Broker Architecture*, estándar del OMG (1991)
 - **Objetos distribuidos**: llamar a métodos remotos
 - Un framework grande: servidores en el medio, infraestructura alrededor
- **Java RMI** — está mucho más cerca de RPC que de CORBA: es cómo Java resuelve RPC
- **REST** — conceptualmente, y en algunos casos, también es una forma de RPC

MOM: *message oriented middleware*

- **RPC es sincrónico:** se llama y quien llamó se bloquea hasta que le responden
- **La alternativa es asincrónica:** se manda el mensaje y quien lo manda sigue con su trabajo
- **Persistencia** — si el destinatario está caído, el middleware guarda y reintenta
- **Discovery** — se manda al canal, sin saber quién va a recibir
- Herederos actuales: **SQS, RabbitMQ, Kafka, Pub/Sub**

Era 4: la era web (2000s)

- Aparece la **banda ancha**: muchos usuarios generando información a gran velocidad
- Y con ella las bases de datos de gran escala, y las empresas que viven de esos datos
- Fueron las primeras en encontrarse con el problema, y las herramientas de entonces no soportaban ese volumen

GFS 2003 • **MapReduce** 2004 • **BigTable** 2006 (*Google*) • **Dynamo** 2007 (*Amazon*)

Era 5: el cloud computing (2006–hoy)

- Las técnicas que esas empresas desarrollaron para sus propios problemas se empiezan a **vender como servicio**
- Igual que la electricidad o el gas: se alquila en lugar de comprar el hardware
- El ejemplo concreto es el almacenamiento: el sistema lo tiene Amazon y se le paga por usarlo

Las eras **4** y **5** son el recorte que estudia esta materia.

**¿POR QUÉ EXISTE
MAPREDUCE?**



Jeff Dean ✓

@JeffDean



Main lore around the origin of MapReduce was that we were rewriting our indexing pipeline for the search system, and we realized that lots of the different phases were conceptually simple but required large scale processing (extract link text from each page, identify language for each page, compute checksum of contents to identify duplicates, etc). Each phase needed to be parallelized, made robust to machine failures, etc. Squinting at each of the phases we came up with MapReduce as an abstraction where we could have an implementation that would do all the complex work under the abstraction boundary, and where the expression of the operations could be nice and simple.

12:02 PM · Feb 28, 2026 · **95K** Views

Transformar y agregar

- Muchos de esos algoritmos tenían todos **la misma forma**: una transformación y una agregación
- Los más complicados se armaban encadenando: transformación, agregación, transformación, agregación
- La salida de un MapReduce puede ser la entrada del siguiente
- A las transformaciones las llamaron **mappers**; a las agregaciones, **reducers**

El sistema de indexación reescrito son **5 a 10 MapReduce encadenados** sobre más de **20 TB** de documentos. Una de sus fases pasó de unas **3.800 líneas de C++** a unas **700**.

El reparto de tareas

- Reconocer **la forma** del algoritmo se separa de la paralelización y distribución.
- Esa parte era muy difícil, o por lo menos necesitaba expertos en sistemas distribuidos
- Y no tenían tantos: por un lado los que escribían algoritmos, por otro los expertos
- MapReduce conecta a los dos: unos escriben map y reduce, el sistema —hecho **una sola vez**— hace el resto

Transparencia, **pero no tanta**: hay que pensar el problema en términos de mappers y reducers.

Dean y Ghemawat



Sanjay Ghemawat (izq.) y Jeffrey Dean • foto de ACM, 2016

MapReduce: Simplified Data Processing on Large Clusters, **OSDI 2004**. Ghemawat firma también **GFS** (2003), el sistema de archivos sobre el que MapReduce se apoya.

Word count: map

```
map(String key, String value):  
    // key: document name  
    // value: document contents  
    for each word w in value:  
        EmitIntermediate(w, "1");
```

Word count: reduce

```
reduce(String key, Iterator values):  
    // key: a word  
    // values: a list of counts  
    int result = 0;  
    for each v in values:  
        result += ParseInt(v);  
    Emit(AsString(result));
```

LA CLAVE INTERMEDIA

El **primer valor del par** que emite el mapper es lo que MapReduce usa para **agrupar**

Los tipos

```
map      (k1, v1)      → list(k2, v2)
reduce (k2, list(v2)) → list(v2)
```

- Map recibe una clave y un valor de ciertos tipos, y emite pares de **otros** tipos
- Esos `k2` y `v2` son exactamente los que recibe reduce: la misma clave, y los valores **ya agrupados**
- La salida de reduce es una lista de valores de ese mismo tipo `v2`

El flujo completo

Tres splits de entrada • tres mappers • una secuencia de pares por cada uno

Todas las apariciones de una misma clave terminan en el **mismo** reduce.

DEL MODELO AL CLÚSTER: EL SHUFFLE

Los mismos nodos cambian de rol

- La imagen simplificada: un grupo de nodos mapper y otro de nodos reducer
- **No es así:** es el mismo clúster, que primero actúa como mapper y después como reducer
- La razón está en los tiempos: hay **dos fases** que no se mezclan
- Con dos clústeres separados, uno quedaría ocioso esperando a que la otra fase termine

M, R y N

- **M** mappers, **R** reducers, **N** nodos físicos: tres cantidades **independientes**
- Un mapper por cada *split* de entrada — mil archivos, o un archivo partido en mil fragmentos, es equivalente
- Es común que **M** $\gg$ **N**, y el paper lo recomienda: nodos siempre ocupados, y redundancia
- M y R son **tareas lógicas**: unidades de trabajo que eventualmente se ejecutan en algún nodo

El shuffle: la redistribución

- Entre las dos fases hay una fase intermedia con nombre propio
- Cada mapper decide, clave por clave, si la manda a R1 o a R2
- Con un dataset normal, de **cada** mapper salen claves para **los dos** reducers
- Ese conjunto de flechas cruzadas es el shuffle

Partir el universo de claves

- El universo de claves intermedias son **las palabras** que aparecen en todos los documentos
- Cada una aparece muchas veces, y todas sus apariciones tienen que ir al **mismo** reducer
- Los que deciden son los **mappers**, y todos tienen que decidir **lo mismo, siempre**

$\text{hash}(k) \% R$

- Se pasa la clave por una función de hash – MD5, SHA: dan un número muy grande que **parece** aleatorio
- Es **determinista**, y su salida está uniformemente distribuida
- A ese número se le aplica módulo **R**, la cantidad de reducers

EL MAPPER Y EL REDUCER POR DENTRO

El mapper por dentro

Con **R = 2**: entra un split, se emite una secuencia de pares, y se escriben **dos archivos locales**

M1_R1 es lo que el mapper 1 prepara para el reducer 1. **M1_R2**, lo que prepara para el reducer 2.

El reducir por dentro

Recibe **tantos archivos como mappers haya**: `M1_R1` , `M2_R1` , `M3_R1`

Y tiene que **agrupar todas las claves**, porque la misma clave aparece en los tres.

Para qué está el sort

- Un sort de todos los archivos que llegan, **juntos**, como si fueran uno solo

```
(a,1) (b,1) (a,1) (c,1) (b,1) → (a,1) (a,1) (b,1) (b,1) (c,1)
```

- Ordenado por clave, las apariciones quedan **contiguas**, y de ahí surge la interfaz que ve el programador: la clave una vez y todos sus valores juntos
- Optimización: Si cada mapper ya emite ordenado, el sort del reducer se vuelve un **merge**

ARQUITECTURA Y LOCALIDAD

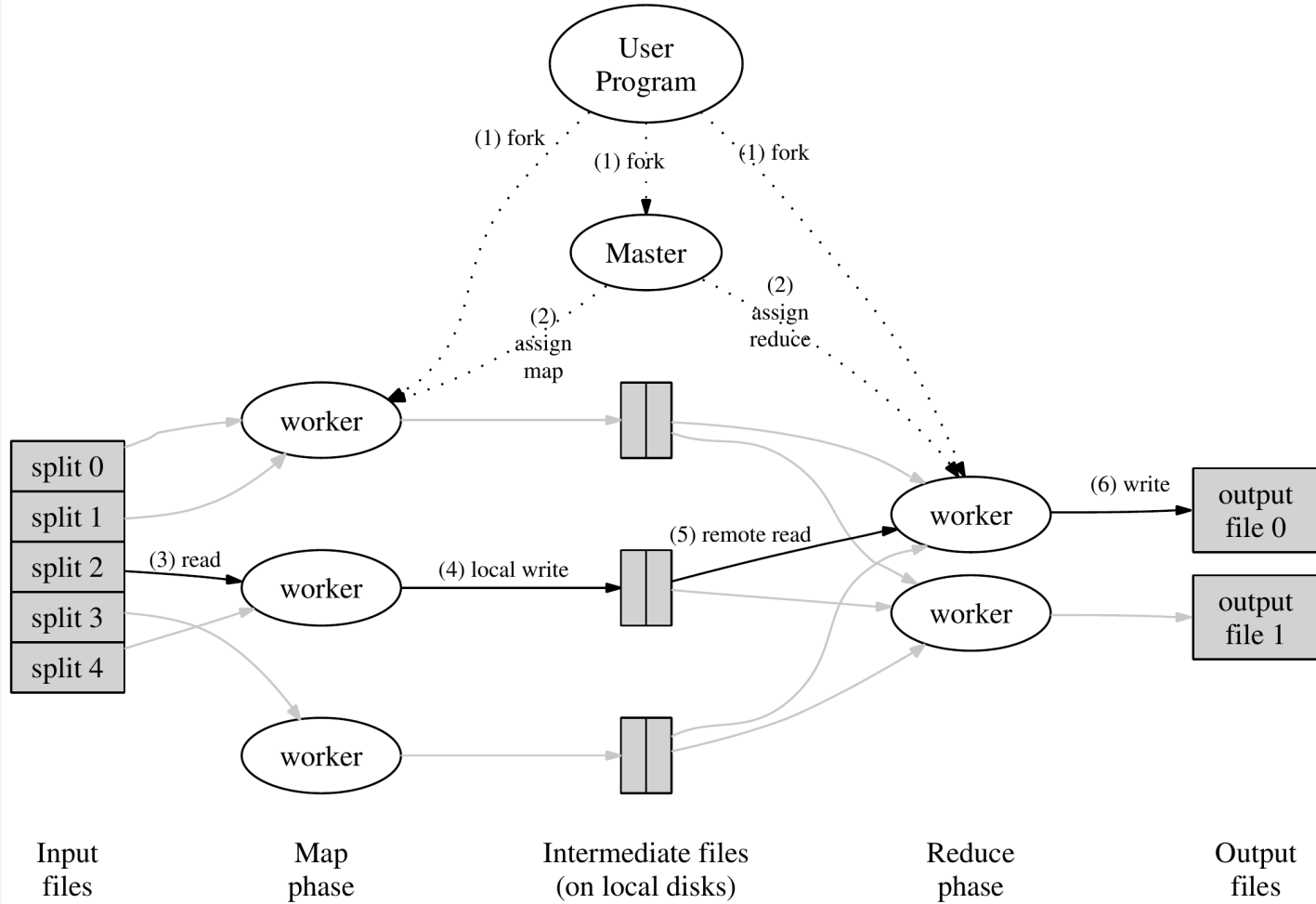


Figure 1: Execution overview

El coordinador y los workers

- Para lanzar un trabajo hay que hablar con el **coordinador**, y es él quien le da trabajo a todos
- Del otro lado, muchos **workers genéricos**
- Y el sentido es el contrario del esperable: **los workers conocen al coordinador**, no el coordinador a los workers
- Cuando un worker arranca, se le pasa la dirección del coordinador y le pregunta: «¿hay trabajo para mí?»

Heartbeats: cada tanto el worker señala que sigue progresando. Si dejan de llegar, esa tarea se le asigna a otro.

El plan en dos fases

- Fase 1: M1, M2, M3 – cada uno con su split de entrada
- Fase 2: R1, R2 – solo cuando **todos** los mappers terminaron

Si un reducer arranca antes de que M3 termine, va a agregar solo las claves de los dos primeros mappers: **se pierden** todas las que faltaban.

La red es el cuello de botella

- Según el paper, el problema principal del sistema era **mover datos**
- **El shuffle es inevitable**: muchas máquinas le tienen que mandar información a muchas otras
- **La lectura de la entrada, en cambio, sí se puede evitar**: no tiene por qué viajar por la red

Co-locación con GFS

- Los nodos tenían un worker de MapReduce **y además** un **chunk server** de GFS
- El chunk server usa los discos locales para guardar fragmentos de **64 MB** de archivos de gran tamaño
- El coordinador consulta **qué chunks tiene ya esa máquina** y le asigna un map que necesite uno de ellos
- La lectura de la entrada pasa a ser una lectura **local** de disco

Es *best effort*: se trata de reducir al máximo la necesidad de ir por la red, no de garantizar que nunca haga falta.

TOLERANCIA A FALLAS

Reintentar lo que falló

- Todo está hecho para que **cada unidad de trabajo se pueda ejecutar más de una vez** sin alterar el resultado
- Un mapper que falla y se re-ejecuta en otra máquina escribe exactamente los mismos archivos
- La estrategia entera es esa: **reintentar las tareas que fallaron**

La condición que lo habilita: **map y reduce tienen que ser deterministas**. Sin aleatoriedad, sin entrada/salida, sin estado.

Ejecución especulativa: *backup tasks*

- Si el coordinador ve un map lento, con el resto de la fase esperándolo, **lanza la misma tarea en otro nodo**
- Las dos compiten: se toma el resultado de la primera que termina y se descarta el trabajo de la otra

Renombrados atómicos

- Del lado del reducer la salida va a **GFS**, que provee **renombrados atómicos**
- El reducer escribe un **archivo temporal** y al terminar lo renombra
- Si hay dos réplicas del mismo reducer, cada una escribe **su** temporal
- La segunda sobrescribe la salida de la primera — y como reduce es determinista, el contenido es el mismo

Se ejecutó dos veces la misma tarea y el resultado final es idéntico.

Falla en el coordinador

- Falla todo: hay que ejecutar el MapReduce nuevamente
- Se puede mitigar **persistiendo el plan** en vez de tenerlo solo en memoria
- Pero por defecto, si el coordinador falla, se reintenta todo

Y no mucho más: el sistema es **simple**, y ahí está buena parte de su valor.

Síntesis

El programador escribe **map** y **reduce**

El sistema resuelve **la distribución**

A cambio exige una sola condición: que las dos funciones sean **deterministas**